

Дизайн и анализ на алгоритми упражнение 6

Станислав Игов (Стъката)

2025/2026

Алчни алгоритми. Maximum Stocks. Make Change. Interval Scheduling.

Алчни алгоритми

Алчен алгоритъм е алгоритъм, който изгражда решение стъпка по стъпка, като на всяка стъпка взима **локално оптималния избор** - избора, който изглежда най-добър в текущия момент, без да разглежда бъдещите последствия.

Ключовото свойство е, че алчният алгоритъм **никога не се връща назад** и не преразглежда вече взети решения. Типичната структура е:

1. **Алчен избор:** Избери локално оптималния елемент спрямо някакъв критерий.
2. **Редукция:** Намали задачата до по-малка подзадача от същия вид.
3. **Итерация:** Повтори докато задачата е решена.

Алчните алгоритми са прости и ефективни, но **не винаги дават оптимален резултат**. Необходимо е коректността им да се доказва.

Доказване на коректност

Има две основни техники за доказване на коректност на алчни алгоритми.

Техника 1: „Алчният остава напред" (Greedy Stays Ahead)

Идеята е да покажем, че на **всяка стъпка** алчното решение е поне толкова добро като всяко оптимално решение, мерено спрямо подходяща мярка.

Структура на доказателството:

1. Дефинирай мярка за сравнение между алчното решение G и произволно оптимално решение OPT .
2. **База:** Алчният избор на стъпка 1 е не по-лош от избора на OPT на стъпка 1.
3. **Индукция:** Ако G е напред след k стъпки, тогава G остава напред след $k + 1$ стъпки.
4. **Заклучение:** Щом G е напред на всяка стъпка, крайното му решение е оптимално.

Типична употреба: Задачи, при които алгоритъмът строи решение стъпка по стъпка и лесно се сравнява с OPT на всяка стъпка - например задача за планиране на интервали (*Interval Scheduling*).

Техника 2: „Размяна на аргументи" (Exchange Argument)

Идеята е да вземем **произволно оптимално решение** OPT и да покажем, че можем да го трансформираме стъпка по стъпка в алчното решение G , без да влошим стойността му.

Структура на доказателството:

1. Приеми, че OPT се различава от алчното решение G .

2. Намери първата точка на разлика между OPT и G .
3. Покажи, че можеш да „размениш“ елемента в OPT с алчния избор, без решението да стане по-лошо.
4. С краен брой такива размени OPT се превръща в $G \Rightarrow$ следователно G е оптимално.

Типична употреба: Задачи, при които е трудно да се дефинира мярка по стъпки, но лесно се аргументира локална размяна - например задачи за оптимална наредба или за купуване на акции.

Задача 1 (Maximum Stocks)

Вход: $C[1 \dots n]$: масив от цени, рационални числа; $L[1 \dots n]$: масив от лимити за количество акции в конкретен ден; k : неотрицателно рационално число (наличен бюджет).

Изход: $\max\{v \mid \text{можем да закупим } v \text{ акции за тези } n \text{ дни}\}$

Разполагаме с k долара и искаме да закупим акции в период от n дни. На ден i цената на акция е $C[i]$, но има ограничение за закупуване само на $L[i]$ акции за този ден. Каква стратегия да следваме, за да закупим максимален брой акции?

```
BuyGreedy(C[1..n] : rationals, L[1..n] : array of naturals, k : non-negative rational)
@1  SortByPricesIncreasing(C[1..n], L[1..n])
@2  stocks <- 0
@3  for i = 1 to n do
@4      stocksToday <- min{k / C[i], L[i]}
@5      stocks <- stocks + stocksToday
@6      k <- k - stocksToday * C[i]
@7  endfor
@8  return stocks
```

Алчна идея: Сортираме дните по цена в нарастващ ред и купуваме максимално количество на най-евтините дни първо. Интуицията е, че с даден бюджет купуваме повече акции при по-ниска цена.

Доказателство за коректност (Exchange Argument)

Твърдение: BuyGreedy връща максималния брой акции, закупими с бюджет k .

Нека след сортирането $C[1] \leq C[2] \leq \dots \leq C[n]$.

Нека $G = (g_1, g_2, \dots, g_n)$ е решението на BuyGreedy, където g_i е броят акции закупени на ден i .

Нека $OPT = (o_1, o_2, \dots, o_n)$ е произволно оптимално решение.

Ще покажем, че $\sum_{i=1}^n g_i \geq \sum_{i=1}^n o_i$.

Ако $OPT = G$ сме готови. Допускаме, че $OPT \neq G$. Нека j е **първият индекс**, на който OPT и G се различават. По конструкцията на BuyGreedy имаме $g_j = \min\left(\frac{k_j}{C[j]}, L[j]\right)$, където k_j е оставащият бюджет преди ден j .

Тъй като j е първата разлика, бюджетите преди ден j в G и OPT съвпадат.

Следователно $o_j < g_j$ — OPT купува строго по-малко акции на ден j .

Тъй като OPT е оптимално и харчи целия бюджет по оптимален начин, той трябва да компенсира: $\exists l > j$ такава, че $o_l > 0$. (Иначе OPT харчи по-малко пари от G при j и не купува повече после, следователно не е оптимално.)

Нека $p = g_j - o_j > 0$ е разликата в броя акции на ден j . Конструираме OPT' , като модифицираме OPT :

- На ден j : купуваме p акции повече, т.е. $o'_j = o_j + p = g_j$.
- На ден l : купуваме q акции по-малко, където $q \cdot C[l] = p \cdot C[j]$, т.е. $q = p \cdot \frac{C[j]}{C[l]}$.

Разходите на OPT' съвпадат с тези на OPT (харчим толкова на ден j , колкото спестяваме от ден l). Тъй като $C[j] \leq C[l]$ (масивът е сортиран и $j < l$), имаме:

$$q = p \cdot \frac{C[j]}{C[l]} \leq p$$

Следователно:

$$\sum_{i=1}^n o'_i = \sum_{i=1}^n o_i + p - q = \sum_{i=1}^n o_i + p \left(1 - \frac{C[j]}{C[l]}\right) \geq \sum_{i=1}^n o_i$$

Тоест OPT' е поне толкова добро като OPT , следователно е оптимално. OPT' съвпада с G на позиции $1, \dots, j$.

Прилагаме аргумента повторно за OPT' . На всяка стъпка намаляваме броя позиции, на които решението се различава от G . След краен брой размени OPT се трансформира в G , без да намалим броя акции. Следователно G е оптимално.

Времева сложност

- Ред 01: Сортиране — $O(n \log n)$.
- Редове 03–07: Цикъл с n итерации, всяка $O(1)$ — $O(n)$.

Следователно $\text{Time}(\text{BuyGreedy}) \in O(n \log n)$.

Задача 2 (Make Change)

Вход: $D[1 \dots n]$: масив от деноминации, подредени в растящ ред; x : положително цяло число.

Изход: $\min\{n \mid \{a_i\}_{i=1}^n \text{ е ресто за } x\}$

Монетна система дефинираме като строго растяща редица от положителни цели числа $\{d_i\}_{i=1}^k$, за която $d_1 = 1$. Казваме, че редицата $\{a_i\}_{i=1}^n$ е **ресто** за x , ако $a_i \in D$ за $1 \leq i \leq n$ и $\sum_{i=1}^n a_i = x$. Искаме минималния брой монети за рестото.

```
MakeChange(D[1..n] : naturals, x : natural)
01  coins <- 0
02  for i = n downto 1 do
03      count <- x / D[i]      // целочислено деление
04      coins <- coins + count
05      x <- x - count * D[i]
06  endfor
07  return coins
```

Алчна идея: Взимаме максимален брой монети с най-голямата деноминация, след това преминаваме към следващата по-малка, и т.н.

Алчният не е винаги оптимален

Алчният подход **не работи за всяка монетна система**. Класически контрапример е монетната система на Римската империя:

Деноминация	Denarius	Sesterius	Dupondius	As	Semis	Quincunx	Triens	Quadrans
Струва X Denarius	1	4	5	10	20	24	30	40

За $x = 8$ Denarius: алчният взима $1 \times 5 + 3 \times 1 = 4$ монети, но оптималното е $2 \times 4 = 2$ монети (два Sesterius-a).

Кога алчният е оптимален?

Алчният алгоритъм е оптимален за **канонични монетни системи**. Достатъчно условие, което лесно се доказва, е геометричната прогресия:

Твърдение: За монетна система от вида $\{1, c, c^2, \dots, c^k\}$, където $c \geq 2$ е естествено число, алчният алгоритъм дава оптимален резултат.

Доказателство (Exchange Argument):

Нека $G = (g_0, g_1, \dots, g_k)$ е алчното решение, където g_i е броят монети с деноминация c^i . Нека $OPT = (o_0, o_1, \dots, o_k)$ е оптимално решение.

Ключово наблюдение. В оптималното решение броят монети с деноминация c^i е $o_i \leq c - 1$, за всяко $i < k$. В противен случай: c монети с деноминация c^i могат да се заменят с 1 монета с деноминация c^{i+1} , намалявайки общия брой.

Единственост на представянето. Стойността x в системата $\{1, c, c^2, \dots, c^k\}$ с ограничение $o_i \leq c - 1$ има **единствено представяне** — то е точно разписът на x в бройна система с основа c . Алчният алгоритъм конструира именно това представяне (взима $g_i = \lfloor x/c^i \rfloor \bmod c$ монети с деноминация c^i).

Следователно $G = OPT$ и алчният е оптимален.

Времева сложност

Цикълът на ред ② се изпълнява n пъти, всяка итерация е $O(1)$.

Следователно $\text{Time}(\text{MakeChange}) \in O(n)$.

Задача 3 (Interval Scheduling)

Студентът стефчо разполага със списък от n мероприятия. Всяко мероприятие има вида (i, s_i, e_i) , където s_i е началният, а e_i е крайният момент. Стефчо иска да посети **максимален брой мероприятия** и при всяко да бъде от началото до края.

Вход: $S[1 \dots n]$, $E[1 \dots n]$, където $(S[i], E[i])$ е времевият интервал на мероприятие i .

Изход: k — максималният брой мероприятия, които стефчо може да посети.

Задачата ни поставя три подзадачи. Ще ги докажем последователно.

Лема 1 (Премахване на доминирани интервали)

Твърдение: стефчо може да премахне от списъка си всяко мероприятие (j, s_j, e_j) , за което съществува мероприятие (i, s_i, e_i) със свойството $[s_i, e_i] \subseteq [s_j, e_j]$. Тоест ако интервал i е *съдържан* в интервал j , тогава j е излишен.

Доказателство (Exchange Argument):

Нека OPT е оптимално решение, което съдържа мероприятието j , но не и i , за което $[s_i, e_i] \subseteq [s_j, e_j]$.

Конструираме OPT' като заменим j с i в OPT .

Тъй като $[s_i, e_i] \subseteq [s_j, e_j]$, всяко мероприятие m , съвместимо с j (т.е. $[s_m, e_m] \cap [s_j, e_j] = \emptyset$), е съвместимо и с i (тъй като i е по-кратко и се съдържа в j). Следователно OPT' е валидно решение.

$|OPT'| = |OPT|$, следователно OPT' е оптимално.

Тоест винаги съществува оптимално решение, което не използва j , и можем да го изтрием от разглеждане.

Лема 2 (Ранно начало = ранен край след почистване)

Твърдение: След премахване на всички доминирани интервали, наредбата по начален момент съвпада с наредбата по краен момент. Следователно алчният алгоритъм, избиращ мероприятиято с **най-ранно начало**, е еквивалентен на алгоритъма с **най-ранен край**.

Доказателство:

Допускаме, че след почистването съществуват два интервала i и j такива, че $s_i < s_j$ (т.е. i започва по-рано), но $e_i > e_j$ (т.е. i завършва по-късно).

Тогава $s_i < s_j \leq e_j < e_i$, следователно $[s_j, e_j] \subset [s_i, e_i]$. Но тогава j доминира i и i трябваше да бъде премахнат — противоречие.

Следователно след почистването: $s_i \leq s_j \Rightarrow e_i \leq e_j$, тоест двете наредби съвпадат.

Алгоритъм

```
IntervalScheduling(S[1..n], E[1..n] : arrays of rationals)
@1  Сортирай интервалите по краен момент E[i] нарастващо
@2  k <- 0
@3  last_end <- -inf
@4  for i = 1 to n do
@5      if S[i] >= last_end then
@6          k <- k + 1
@7          last_end <- E[i]
@8      endif
@9  endfor
@10 return k
```

Алчна идея: Сортираме по краен момент. На всяка стъпка избираме следващото съвместимо мероприятие с най-ранен край (еквивалентно на най-ранно начало след почистването по Лема 2).

Лема 3 / Доказателство за коректност (Greedy Stays Ahead)

Твърдение: IntervalScheduling връща максималния брой взаимно съвместими интервали.

Нека интервалите са сортирани по краен момент: $e_1 \leq e_2 \leq \dots \leq e_n$.

Нека $G = (i_1, i_2, \dots, i_p)$ са индексите на интервалите, избрани от алчния (в реда на избор), и нека $OPT = (j_1, j_2, \dots, j_q)$ са индексите на произволно оптимално решение (също сортирани по краен момент).

Дефинираме **мярка за сравнение**: след t избора алчният е „напред ако $e_{i_t} \leq e_{j_t}$ — крайт на t -тия му избор е не по-голям от края на t -тия избор на OPT .

База ($t = 1$). Алчният избира интервала с минимален краен момент сред всички. Следователно $e_{i_1} \leq e_{j_1}$. Мярката е изпълнена.

Индукционна стъпка. Допускаме, че след t стъпки $e_{i_t} \leq e_{j_t}$. Ще докажем, че $e_{i_{t+1}} \leq e_{j_{t+1}}$.

Интервалът j_{t+1} е съвместим с j_t , следователно $s_{j_{t+1}} \geq e_{j_t} \geq e_{i_t}$. Тоест j_{t+1} е съвместим и с i_t — и следователно е кандидат за $(t + 1)$ -вия избор на алчния.

Алчният обаче избира интервала с **минимален краен момент** сред съвместимите, поради което:

$$e_{i_{t+1}} \leq e_{j_{t+1}}$$

Заклучение. По индукция $e_{i_t} \leq e_{j_t}$ за всяко $t \leq \min(p, q)$.

Допускаме, че $p < q$. Тогава след p избора алчният спира, но OPT има още поне един интервал j_{p+1} . Интервалът j_{p+1} е съвместим с j_p , следователно $s_{j_{p+1}} \geq e_{j_p} \geq e_{i_p}$. Тоест j_{p+1} е съвместим с i_p — алчният би трябвало да го е включил. Противоречие.

Следователно $p \geq q$, а тъй като $q = |OPT|$ е максималното, получаваме $p = q$ и алчният е оптимален.

Времева сложност

- Ред @1: Сортиране — $O(n \log n)$.
- Редове @4–@9: Цикъл с n итерации, всяка $O(1)$ — $O(n)$.

Рекурентното уравнение не е приложимо (не е рекурсивен алгоритъм). Общата сложност се доминира от сортирането:

$$\text{Time}(\text{IntervalScheduling}) \in O(n \log n)$$